ChatGPT

Executive Summary

We propose building **“Common App Pakistan”**, a production-grade clone of the Common Application platform tailored for Pakistani universities. Like CommonApp (which aggregates applications for 1,000+ colleges [1]), our system will let **students** create one profile and apply to multiple universities, while **university admins** review applications via dashboards. The spec strictly locks the tech stack (React/Next.js frontend, Node/Express backend, PostgreSQL DB, Prisma optional) and defines atomic phases and tasks to ensure a deterministic implementation.

Key flows include student signup/login, profile creation, college search, multi-university application submission, and dual dashboards (student and admin). We enforce exact UI/UX replication of Common App’s structure: e.g. a **Dashboard** showing progress [2], a **“My Common Application”** multi-section form [3] (personal info, family, education, tests, activities, essays), an **Explore/College Search** page with filters [4] [5], and **My Colleges** status tracking. Auth is email/password (with JWT or sessions) and roles include Student, University Admin, and Superadmin. We support English/Urdu (with RTL layouts for Urdu [6]) and follow accessibility best practices.

All deliverables will be broken into strict, testable tickets. Each task includes acceptance criteria, test steps, and example data. We include tables of pages/components, API endpoints, and a phased ticket list. A mermaid ER diagram defines the data model, and a mermaid Gantt chart shows the timeline. Failure to follow the exact spec (adding fluff, skipping details, or deviating from requirements) will be flagged as failure. The output format and execution rules mirror a senior-engineer prompt to a development agent.

Project Goal and Tech Stack

- **Goal:** Build a **production-ready** Common App clone for Pakistan. Students fill one profile and apply to many universities; universities manage and review applications. The app must be fully functional, deployed on Replit.
- **Locked Tech Stack:** React (or Next.js) frontend; Node.js + Express backend; PostgreSQL database (Prisma ORM optional); Email/password auth (with secure hashing and JWT/session); Styling with Tailwind CSS; Hosted on Replit with environment variables.
- **Auth Model:** Email & password sign-up/login; hashed passwords (e.g. bcrypt); JWT or express-session for protected routes; email verification on signup (users must verify before applying).
- **User Roles:**
- **Student:** Can sign up, create/edit profile, search/apply to universities, view application status.
- **University Admin:** Has an account linked to one or more universities; can create/edit “University” records (name, location, requirements, deadlines) and review applications for those universities.
- **Superadmin:** Can manage universities and admin accounts globally (optional; can also seed initial data).

Core Flows

1. **Signup/Login:** Students (and admin/superadmin) create accounts (email, password) and log in. Email verification is required before full access.
2. **Student Profile:** After login, students complete **My Common Application** (multi-page form) including personal info, family background, education history, test scores, extracurriculars, and essays (mirroring Common App sections [3]). Each section has required fields and validations.
3. **University Management:** Superadmin or admin creates universities with fields (name, type, location, deadlines, essay prompts/requirements). University admins can update these details and set application deadlines.
4. **College Search:** Students browse a searchable/filterable list of universities (the "Explore Colleges" page [4] with filters: first-year, transfer, size, setting, etc) and view details of each.
5. **Application Flow:** Students add colleges to their "My Colleges" list and submit applications. Each application includes the student's profile data (auto-filled), any college-specific questions, and uploaded documents (transcripts). Application status is tracked (e.g. Not Submitted, Submitted, Reviewed, Admitted/Rejected).
6. **Dashboards:**
7. **Student Dashboard:** After login, shows progress (e.g. "21/72 sections complete") and lists college applications with statuses [2]. Provides quick links to continue profile or manage applications.
8. **Admin Dashboard:** University admins see incoming applications, filter/sort them, and update statuses or request documents.

UI Pages & Components

We replicate Common App's UI precisely. Below is an inventory of pages/components. Validation rules use standard patterns (e.g. required fields, email format, password strength). Responsive design uses Tailwind's default breakpoints (`sm` 640px, `md` 768px, `lg` 1024px, etc). All forms are labeled and support accessibility.

Page/Component	Description / Key Fields
Landing / Home	Marketing content; "Sign In / Create Account" links (commonapp.org style header). Simple English/Urdu toggle.
Signup (Create Account)	Tabs for "Student" (and "Transfer", if applicable) sign-up. Fields: Email*, Password* (min 8 chars), Confirm PW*.
Login	Fields: Email*, Password*; link "Forgot Password". After login, redirect to Dashboard.
Email Verification	Page instructing user to check email, with link to verify token.
Student Dashboard	Sidebar nav: Dashboard (active), My Common App, My Colleges, Explore, Settings. Shows progress bars (e.g. "21/72 sec.") [2], list of colleges.

Page/Component	Description / Key Fields
My Common Application	Multi-step form with sidebar sections (Profile, Family, Education, Testing, Activities, Writing, Courses & Grades). For each section: form fields with Save button, progress checkmarks [3] .
– Personal Info	Fields: First Name*, Preferred Name (optional), Middle, Last Name*, Suffix, Former Name? (yes/no), DOB*, Gender*, Citizenship, CNIC/Passport, Contact Info.
– Family	Yes/No: Parents married, reside with, siblings. Parent 1/2: Name, Living (yes/no), Email/Phone (optional), Education level, Occupation. Siblings: list of name/age/grade.
– Address	Fields: Current Address*, City*, Province*, Country*, Phone*.
– Demographics	Ethnicity (options), First-gen college (yes/no), etc.
– Education	Schools Attended (add row: school name, city, country, years, diploma type), High school grades (GPA, scale), Current 12th-grade courses with grades.
– Testing	List tests (SAT, ACT, etc): yes/no, if yes add test type, date, score.
– Activities	"Do you have extracurriculars?" yes/no. If yes, add items: Name, Category, Position, Hours/week, Weeks/year, Description.
– Writing (Essay)	Personal Essay: textarea (min 250 words). Additional Info: textarea (optional). Upload: ability to attach file (PDF/docx).
Explore / College Search	Filters: First-year, Transfer, Campus Setting, Location (Country/Province), Financial Aid (Yes/No), Minority-serving, Type (Public/Private), Size. Sort options. Grid view of colleges with logo, name, basic info. [4]
College Detail	University name, logo, description, requirements (application deadline, needed documents, essay prompts). "Add to My Colleges" button.
My Colleges (Applications)	List of colleges student has added. Columns: College name, Status (Not Started, In Progress, Submitted, Decision), Deadline. Buttons: "Continue Application" or "Submit". Section for Recommenders/FERPA (optional future feature).
University Admin Dashboard	Sidebar: Dashboard, Applications, Profile, Settings. Main view: Table of applications (student name, applied on date, status). Filters by status, search by name/email.
Application Review (Admin)	View a student's submitted application: shows all profile sections (read-only), uploaded files. Dropdown to update status (e.g. "Under Review", "Accepted", "Rejected") and leave comments.
University Management (Admin)	Form to create/edit University: Name*, Location*, Type*, Min/Max GPA/Marks*, Requirements (e.g. required fields from student app), Deadline* (date).
Superadmin Pages	(If implemented) List of universities, list of admin users, ability to assign admins.

Page/Component	Description / Key Fields
Modals/Alerts	Confirmation dialogs (e.g. “Are you sure you want to submit? Once submitted you cannot edit.”). Success/error toasts.
Navigation & Layout	Responsive header with logo and nav links; sidebar on desktop, collapsible menu on mobile. All forms use accessible labels and error messages. Urdu text triggers RTL layout [6].

Data Model

The data model includes **User**, **StudentProfile**, **University**, and **Application** (plus related sub-records). We provide a mermaid ER diagram and a Prisma schema snippet. This ensures one-to-many (students → applications, universities → applications) and one-to-one/profile relationships.

```
erDiagram
    USER ||--o{ STUDENT_PROFILE : has
    USER ||--o{ APPLICATION : submits
    UNIVERSITY ||--o{ APPLICATION : receives

    USER {
      int id PK
      string email
      string passwordHash
      enum role         "student, uni_admin, superadmin"
      datetime createdAt
      datetime updatedAt
    }
    STUDENT_PROFILE {
      int user_id PK
      string firstName
      string lastName
      string preferredName?
      date   dob
      string gender
      string address
      string city
      string province
      string country
      string phone
      string cnic?     "Pakistan CNIC or passport"
      string nationality
      string demographics? "JSON: ethnicity, first-gen, etc"
      text   essay
      text   additionalInfo
      datetime updatedAt
    }
```

```
    UNIVERSITY {
      int id PK
      string name
      string city
      string province
      string country
      enum type        "public, private"
      string description
      date   applicationDeadline
      string requirements "e.g. required GPA, etc"
      int    createdBy FK to USER
    }
    APPLICATION {
      int id PK
      int studentId FK to USER
      int universityId FK to UNIVERSITY
      enum status      "not_started, in_progress, submitted, accepted, rejected"
      datetime submittedAt?
      datetime updatedAt
    }
```

Prisma Schema (example):

```
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  password  String
  role      String   // 'student', 'uni_admin', 'superadmin'
  profile   StudentProfile?
  applications Application[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model StudentProfile {
  userId       Int    @id
  firstName    String
  lastName     String
  preferredName String?
  dob          DateTime
  gender       String
  address      String
  city         String
  province     String
  country      String
  phone        String
```

```
  cnic         String?
  nationality  String
  demographics Json?
  essay        String
  additionalInfo String?
  user         User   @relation(fields: [userId], references: [id])
}

model University {
  id           Int      @id @default(autoincrement())
  name         String
  city         String
  province     String
  country      String
  type         String
  description  String
  deadline     DateTime
  requirements String
  createdById  Int
  createdBy    User     @relation(fields: [createdById], references: [id])
  applications Application[]
}

model Application {
  id           Int       @id @default(autoincrement())
  studentId    Int
  universityId Int
  status       String
  submittedAt  DateTime?
  updatedAt    DateTime  @updatedAt
  student      User       @relation(fields: [studentId], references: [id])
  university   University @relation(fields: [universityId], references: [id])
}
```

API Endpoints

Below is a table of REST endpoints (Node/Express JSON API). All request/response bodies use JSON. Authentication (via JWT/session) is required for protected routes. Example request/response snippets illustrate payloads. The auth token is sent in the `Authorization: Bearer <token>` header.

Method	Endpoint	Auth	Request Body (JSON)	Response (JSON)
POST	`/api/auth/signup`	No	`{ "email": "student@example.com", "password": "P@ssw0rd" }`	`{ "message":"User created. Verify email." }`
POST	`/api/auth/login`	No	`{ "email": "student@example.com", "password": "P@ssw0rd" }`	`{ "token": "jwt_token", "user": {id, email, role} }`
GET	`/api/auth/verify-email?token=<token>`	No	–	`{ "message": "Email verified" }`
GET	`/api/profile`	Yes (Student)	–	`{ profile: {firstName, lastName, ...} }`
PUT	`/api/profile`	Yes (Student)	`{ firstName: "Ali", lastName: "Khan", dob: "2005-01-01", ... }`	`{ profile: { ...updated fields... } }`
POST	`/api/profile/education`	Yes (Student)	`{ schoolName: "High School", city: "...", startYear: 2021, endYear: 2023 }`	`{ educationRecord: {id, ...} }`
PUT	`/api/profile/education/:id`	Yes (Student)	`{ schoolName: "...", ... }`	`{ educationRecord: {id, ...} }`
DELETE	`/api/profile/education/:id`	Yes (Student)	–	`{ "message": "Deleted" }`
(Similar endpoints for activities, testing, etc.)				

Method	Endpoint	Auth	Request Body (JSON)	Response (JSON)
GET	`/api/universities`	Yes	–	`{ universities: [ {id, name, city, province, deadline, ...} ] }`
GET	`/api/universities/:id`	Yes	–	`{ university: {id, name, ..., requirements } }`
POST	`/api/universities`	Yes (Admin)	`{ name: "ABC University", city: "Lahore", province:"Punjab", country:"PK", type:"public", deadline:"2026-07-31", requirements:"GPA>70%" }`	`{ university: {...} }`
PUT	`/api/universities/:id`	Yes (Admin)	`{ name: "Updated Name", deadline: "2026-08-15", ... }`	`{ university: {...} }`
DELETE	`/api/universities/:id`	Yes (Admin)	–	`{ "message": "Deleted" }`
POST	`/api/applications`	Yes (Student)	`{ studentId: 1, universityId: 5 }`	`{ application: {id, status:"in_progress", ...} }`
PUT	`/api/applications/:id`	Yes (Student)	`{ status: "submitted", submittedAt: "2026-05-10T12:00:00Z" }`	`{ application: {...} }`
GET	`/api/applications`	Yes (Student)	–	`{ applications: [ {id, university: {name,...}, status, submittedAt} ] }`
GET	`/api/applications/:id`	Yes (Student/ Admin)	–	`{ application: {id, student: {...}, university: {...}, status, ...} }`

Method	Endpoint	Auth	Request Body (JSON)	Response (JSON)
PUT	`/api/applications/:id/status`	Yes (Admin)	`{ status: "accepted" }`	`{ application: {id, status:"accepted"} }`
POST	`/api/upload`	Yes	Multipart/form-data: file upload (e.g. transcript PDF)	`{ "url": "uploads/filename.pdf" }`

Note: All `:id` paths are numeric IDs. Example requests and responses above illustrate typical JSON formats.

Routing & File Structure

We will use standard file-based routing (if Next.js) or React Router (if plain React) with paths matching pages above. For example:

- `/login`, `/signup`, `/verify-email`, `/dashboard`, `/profile`, `/my-colleges`, `/explore`, `/university/[id]`, `/admin/universities`, `/admin/applications`, etc.

A typical project tree (showing new/changed files) might be:

```
/src
  /components
    Navbar.js
    Sidebar.js
    FormInput.js
    CollegeCard.js
    ...
  /pages
    /auth
      Login.jsx
      Signup.jsx
      VerifyEmail.jsx
    Dashboard.jsx
    Profile.jsx
    MyColleges.jsx
    Explore.jsx
    UniversityDetail.jsx
    /admin
      Universities.jsx
      UniversityForm.jsx
      Applications.jsx
```

```
    ApplicationDetail.jsx
  Settings.jsx
/api
  auth.js
  profile.js
  universities.js
  applications.js
  upload.js
prisma/
  schema.prisma
```

Styling & Internationalization

- **Styling:** Use Tailwind CSS for utility-first styling. Responsive breakpoints: `sm` (≤640px, mobile), `md` (≤768px, tablet), `lg` (≥1024px, desktop). Forms should be mobile-friendly. Color palette can follow Common App's blue/gray theme.
- **Accessibility:** All forms use `<label>`s, appropriate `aria-*` attributes on modals and error messages, and semantic HTML. Buttons have `aria-pressed` or role if toggles. Ensure color contrast and keyboard navigation.
- **Internationalization:** Support English and Urdu. Use a library like i18next or Next.js i18n. Detect language toggle. Urdu layout is right-to-left: text and UI mirroring for RTL scripts [6]. Use CSS `dir="rtl"` for Urdu. All strings pulled from language files.

Testing Strategy

- **Unit Tests:** Use Jest (or Vitest) and React Testing Library for frontend components (forms, nav, utility functions). Test component rendering and form validation.
- **Integration Tests:** Supertest (or similar) to test Express API routes (signup, profile CRUD, application flows). Use a test database (SQLite or a Postgres test instance) seeded with known data.
- **E2E Tests:** Use Cypress or Playwright for end-to-end tests: simulate user flows (signup → profile fill → add college → submit → admin review). Verify UI flows match expected screenshots and behavior.
- **CI:** Configure GitHub Actions to run linters (`eslint`, `prettier`), tests, and code quality on push. Ensure at least 80% test coverage.
- **Example Test Data:** Use realistic dummy data (student names, university names). E.g. Student: { email: "ali.khan@mail.com", password: "Secret123!" }; University: {name: "Lahore Tech Univ", deadline: "2026-07-31"}.

Deployment (Replit)

1. **Replit Environment:** Use Replit's Node.js template. In Replit "Secrets", set `DATABASE_URL` (PostgreSQL connection string) and `JWT_SECRET`. Optionally use a free Postgres host (Heroku, Supabase, ElephantSQL).
2. **Database:** Use Prisma to manage migrations. On deploy, run `npx prisma migrate deploy` or `prisma db push` to create tables. (Replit can run `npm run migrate` on startup).

3. **Build & Start:** Configure `package.json` scripts: `"start": "node src/server.js"`, etc. Replit will run the start command. Ensure frontend build (if Next.js) or React build is produced (if using CRA).
4. **Port:** Replit assigns `process.env.PORT`; ensure Express listens on `process.env.PORT`.
5. **Continuous:** Any push to Replit triggers redeploy; check database connectivity.
6. **Testing in Replit:** After deployment, manually test key flows (signup, create profile, apply) and run automated tests via terminal.

Strict Prompt Format for Agent

In all tasks below, the output format for the Replit agent is mandatory and non-negotiable. The agent's responses must adhere exactly to the structure: **(1) WHAT YOU ARE BUILDING (1–2 lines), (2) FILE TREE (only changed/new files), (3) FULL CODE (complete), (4) HOW TO TEST (steps)**. Any deviation (fluff, partial code, renaming features, etc.) is considered a failure. Each task is atomic; do not jump ahead. If a requirement is unclear, the agent must ask questions.

Below is the **strict copy-paste prompt** for the Replit agent:

```
You are not here to brainstorm or improvise.
You are acting as a senior production engineer working on a real, deployable
app.

Your job: EXECUTE EXACTLY what is asked in each ticket. Nothing more, nothing
less.
If you do anything outside the specified scope, it is a failure.

---------------------------------
PROJECT CONTEXT
---------------------------------
We are building a production-ready application modeled on the Common
Application, adapted for use in Pakistan.
Students create one profile (including personal info, education, test scores,
essays) and can apply to multiple universities.
Universities and their admins manage requirements, deadlines, and review student
applications.
Universities include details like name, location, type, application deadline,
and required fields.
Universities may offer programs; superadmin can seed initial data and manage
overall settings.

---------------------------------
LOCKED TECH STACK
---------------------------------
- Frontend: React (use Next.js if SSR needed) with Tailwind CSS.
- Backend: Node.js + Express.
- Database: PostgreSQL (use Prisma or SQL directly).
```

```
- Authentication: Email/password (bcrypt for hashing), JWT or sessions for auth.
- Hosting: Replit environment (use environment variables for secrets and DB
URL).

----------------------------------
AUTH MODEL & ROLES
----------------------------------
- Roles: Student, UniversityAdmin, Superadmin.
- On signup, default role=Student. UniversityAdmin accounts must be created by
Superadmin or an existing UniversityAdmin.
- Secure authentication: require email verification before full access.
- Protect routes: only Students can access student routes, only UniversityAdmins
can manage university data and applications for their universities.

----------------------------------
CORE FLOWS (build in order)
----------------------------------
PHASE 1: AUTH SYSTEM
- Ticket: Build DB schema (users table, roles, profiles) [7].
- Ticket: Signup API (POST /api/auth/signup).
- Ticket: Login API (POST /api/auth/login).
- Ticket: Session/Auth middleware (verify JWT).
- Ticket: Email verification flow.
- Ticket: Signup/Login frontend forms (React).

PHASE 2: STUDENT PROFILE
- Ticket: StudentProfile API & models (personal, address, family info).
- Ticket: Education model/API (education history).
- Ticket: Testing & Activities model/API.
- Ticket: Essay section model/API.
- Ticket: React UI for each profile section (forms with validation).

PHASE 3: UNIVERSITY SYSTEM
- Ticket: University model (fields: name, location, type, deadline,
requirements).
- Ticket: University CRUD API (Create/List/Update/Delete) [4].
- Ticket: University Admin account logic.
- Ticket: React UI for University creation/edit (admin).
- Ticket: University detail page for students (read-only view).

PHASE 4: APPLICATION FLOW
- Ticket: Application model/API (link student↔university, status).
- Ticket: “Add University” action (student posts application).
- Ticket: File upload endpoint (for transcripts, essays).
- Ticket: Application submission and status update APIs.
- Ticket: React UI for MyColleges list and apply actions.
- Ticket: React UI for Application review (admin updates status).
```

```
PHASE 5: DASHBOARDS & COMPLETION
- Ticket: Student Dashboard UI (progress and list of applications) [2].
- Ticket: Admin Dashboard UI (list applications, filter, search).
- Ticket: Internationalization (Urdu/English toggle, RTL layout) [6].
- Ticket: Accessibility checks.
- Ticket: Testing setup (unit, integration, E2E).
- Ticket: Deployment setup for Replit.

----------------------------------
OUTPUT FORMAT (MANDATORY)
----------------------------------
For each ticket, your response must follow exactly:

1. **WHAT YOU ARE BUILDING** (1–2 lines)
2. **FILE TREE** (only the changed/new files, with hierarchy)
3. **FULL CODE** (complete code for those files, no omissions)
4. **HOW TO TEST** (exact steps to verify functionality)

Any missing part or deviation makes the answer INVALID.

----------------------------------
FAIL CONDITIONS
----------------------------------
Your response is WRONG if you:
- Skip implementing required code or features.
- Output explanations or commentary.
- Change any requirement or add new features not asked.
- Provide partial code (use complete, executable code).
- Do not follow the output format exactly.

----------------------------------
START NOW
----------------------------------
```

Phased Task List

Below is a prioritized list of atomic tickets (with acceptance criteria, test steps, and example data). Each ticket corresponds to one item in **CORE FLOWS** above.

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
DB Schema Design	1	- Tables for User (id, email unique, passwordHash, role, timestamps). - Table StudentProfile (FK to User, firstName, lastName, etc). - Table University, Application as per ER.	1. Write Prisma models or SQL DDL. 2. Run migrations. 3. Check DB: tables and columns exist with correct types and constraints.	N/A
Signup API	1	- **POST** `/api/auth/signup` accepts email, password. - Creates User (role=student) and StudentProfile. - Sends verification token (simulate or log). - Returns success message.	1. Send POST with valid payload. 2. Expect 200 and user in DB (password hashed). 3. Verify `role` is “student” and profile row created. 4. Ensure no password in response.	`{email:"test@mail.com", password:"Secret123"}`
Login API	1	- **POST** `/api/auth/login` accepts email, password. - Verifies credentials and returns a JWT token. - Returns user info (id, email, role).	1. Send valid credentials. 2. Expect token in response. 3. JWT decodes to correct user id and role. 4. Invalid creds -> 401 error.	Use user from Signup test.
Session Middleware	1	- Protect routes: invalid/no token returns 401. - Valid token allows access. - Token must carry role.	1. Call a protected endpoint without `Authorization` header. Expect 401. 2. With token from login, expect access (200). 3. Check req.user is set correctly.	N/A

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
Student Profile API	2	- **GET/ PUT** `/api/profile` returns/updates profile for authenticated student. - Fields: firstName*, lastName*, dob*, address, city, province, etc. Required fields enforced.	1. Log in as student. 2. GET `/api/profile`: expect empty/ default. 3. PUT with valid data. Expect 200 and updated. 4. GET again to see changes. 5. PUT with missing required -> 400.	`{firstName:"Ali", lastName:"Khan", dob:"2005-01-01", address:"...", city:"Lahore", province:"Punjab", country:"PK"}`
Education API	2	- CRUD endpoints under `/api/profile/education`. - Can add, list, update, delete education records. - Each record: schoolName*, city*, startYear*, endYear*.	1. POST new education. Expect 201 and returned object with ID. 2. GET list: see new record. 3. PUT update a field: verify changed. 4. DELETE record: expect removal.	`{schoolName:"Govt High School", city:"Lahore", startYear:2021, endYear:2023}`
Activities API	2	- Similar CRUD under `/api/profile/activities`. - Each activity: name*, category, hours/week, weeks/year, description.	1. POST activity. 2. GET all: record present. 3. Edit and delete, verifying each.	`{name:"Basketball", category:"Sport", hoursPerWeek:5, weeksPerYear:10, description:"School team captain."}`
Essay API	2	- **GET/PUT** essay text and additional info at `/api/profile/essay`. - Essay required min 250 chars (validate). - Additional info optional.	1. PUT essay with <250 chars: expect validation error. 2. PUT valid essay. 3. GET essay: text matches.	`{essay:"A".repeat(300), additionalInfo:"Interested in physics and maths."}`

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
Signup Form UI	1	- React signup form with fields (email*, password*, confirm password*). - Shows validation errors (e.g. password length, mismatch). - On submit, calls signup API and handles response.	1. Load Signup page in browser. 2. Try submitting empty: errors for required fields. 3. Enter invalid email: validation error. 4. Enter valid and submit: see success (or handle API response).	Same as Signup API example.
Login Form UI	1	- React login form (email*, password*). - On success, saves token (e.g. in cookie/localStorage) and redirects to Dashboard. - On failure, displays error.	1. Load Login page. 2. Submit blank: errors. 3. Enter wrong creds: see error message. 4. Enter valid creds (from Signup): should redirect to Dashboard.	Use credentials created above.
Profile UI (Personal)	2	- Form for personal info (as listed above). - Required fields validated. - Pre-fills from API if existing. - “Save” button updates via API.	1. After login, navigate to Profile. 2. Enter sample data and save. 3. Check success message. 4. Refresh page: data persists (GET from API). 5. Leave required field blank: see validation error.	Use StudentProfile API data.
Education UI	2	- UI to add multiple education entries (table or cards). - Fields match API. - Can add/edit/remove rows, saving to backend.	1. In profile page, navigate to Education section. 2. Add a school, save. 3. Entry appears in list. 4. Edit it, save. 5. Delete it.	Match Education API data.

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
Activities UI	2	- Similar to Education UI for extracurriculars. - Add, edit, delete entries with validations.	1. In Activities section, add an activity with all fields. 2. Save and verify it appears. 3. Edit hours, save. 4. Delete and verify removal.	Use Activities API data example.
Essay UI	2	- Textarea for essay (min 250 chars) and optional additional info. - Shows live character count or warning. - Save persists to backend.	1. Enter short essay (<250): see “too short” error. 2. Enter long essay: no error. 3. Save and verify success. 4. Refresh: essay text remains.	`{essay: "...(300 chars)...", additionalInfo:"..."}`
University CRUD API	3	- CRUD endpoints under `/api/universities`. - Admin can create, edit, delete universities. - University fields validated (name*, location*, deadline*).	1. POST new university as admin. Expect 201 with id. 2. GET list: see new university. 3. PUT update name, verify. 4. DELETE and ensure removal.	`{name:"Test University", city:"Karachi", province:"Sindh", country:"PK", type:"public", deadline:"2026-07-01", requirements:"GPA70+"}`
University Form UI	3	- Admin page to add/edit university details (fields as above). - Validates required fields. - On success, redirects to list.	1. As admin, navigate to “Add University” page. 2. Try submitting empty: errors. 3. Fill valid data, submit. 4. Ensure new uni appears on admin list page. 5. Test editing an existing university.	Use University CRUD API data.

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
University List (Explore) UI	4	- Public page listing all universities (cards or table). - Supports filtering by first-year (all in this context), location, type, etc. - “Grid view” shows name and info [4] .	1. Visit Explore page. 2. Verify universities created appear. 3. Use filter (e.g., type=public) and confirm results narrow.	Filters: `{type:"public"}`.
Add Application API	4	- **POST** `/api/applications` creates an application (status “in_progress”). - StudentId from token, supply universityId. - Cannot add same university twice.	1. As student, POST with own id and a universityId. 2. Expect 201 and status “in_progress”. 3. POST again same uni: expect error (400). 4. GET `/api/applications`: see new entry.	`{studentId:1, universityId: 5}`
File Upload API	4	- **POST** `/api/upload` accepts multipart file (PDF/doc). - Returns stored file path/URL. - File stored in server (e.g., /uploads).	1. Use a tool or form to upload a sample PDF. 2. Expect 200 with `{url: "uploads/filename.pdf"}`. 3. Check file exists in server directory.	Upload dummy PDF file.
Submit Application API	4	- **PUT** `/api/applications/:id` with `{status:"submitted", submittedAt: now}`. - Changes status to “submitted” (no further edits allowed).	1. POST application (in-progress). 2. PUT with status=submitted. 3. GET application: status is “submitted”. 4. Ensure trying to PUT again (or edit profile) returns error.	Use application created above.

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
Admin Review UI	4	- In University Admin dashboard, each application shows student name, status, and a “Review” button. - Clicking shows full application details. - Dropdown to set status (Accepted/Rejected) and save.	1. As admin, go to Applications page. 2. See the student who applied. 3. Click “Review” and change status to Accepted. 4. Save and check status in list updated.	Application from previous test.
Student Dashboard UI	5	- Upon login, redirect to Dashboard. - Shows "My Common App" progress (e.g. “X/total sections complete”) [2] . - Shows quick link to incomplete sections. - Shows list of MyColleges with statuses.	1. Login with student who has some profile/app data. 2. Verify dashboard shows correct progress count. 3. List of colleges shows added applications and statuses.	Student with one submitted application.
Admin Dashboard UI	5	- Upon admin login, show list of applications for their university. - Filter by status. - Quick stats (e.g. total applications).	1. Login as UniversityAdmin. 2. Dashboard lists applications (from test). 3. Use status filter to hide/show entries.	Admin of the university applied to above.
Language Toggle (i18n)	5	- A toggle/button switches between English and Urdu. - All text translates; layout shifts RTL for Urdu [6] .	1. Click “اردو” toggle. 2. Verify UI text changes to Urdu. 3. Inspect DOM: `dir="rtl"` on `<html>` or main container. 4. All text alignment reverses.	N/A

Ticket	Phase	Acceptance Criteria	Test Steps	Example Data
Unit & E2E Tests	5	- Jest unit tests for key functions (profile validation, reducers). - React Testing Library for form components. - Cypress tests: sign-up, create profile, apply to university, admin review.	1. Run `npm test`: all unit tests pass. 2. In Cypress, automate login/signup and major flows, all should pass. 3. Check coverage report >80%.	Use test credentials created above.
Deploy on Replit	5	- Application runs on Replit with `npm start`. - DB migrations applied. - Env variables set (`DATABASE_URL`, `JWT_SECRET`). - Visiting Replit URL shows login page.	1. In Replit console, run migrations. 2. Start server, visit URL. 3. Perform a quick smoke test (signup/login). 4. Verify DB tables exist via Prisma Studio or query.	(Use Replit's env and a test DB.)

In summary, each ticket above is an atomic task. Follow **exactly** the acceptance criteria and format for deliverables. Test each step thoroughly with the provided example data to ensure correctness.

The above spec, tables, diagrams, and task list encapsulate a comprehensive, step-by-step implementation plan for the Common App Pakistan clone, with all details locked for deterministic execution.

---

1 Apply to college with Common App | Your future starts here
https://www.commonapp.org/

2 3 Common App user interface updates 2025-2026
https://www.commonapp.org/files/Common-App-UI-updates.pdf

4 5 Explore colleges using Common App | Common App
https://www.commonapp.org/explore/

6 Planning for RTL Languages: How Layout, Content, and QA Fit Together | Argos Multilingual
https://www.argosmultilingual.com/blog/planning-for-rtl-languages-how-layout-content-and-qa-fit-together

7 resources.finalsite.net
https://resources.finalsite.net/images/v1724443149/fultonschoolsorg/tiftfmod2ys0txaa74nc/24-25CommonAppStudentFAQsPDF.pdf